

# **Tratamento de erros e *design* de respostas adequadas**

Programação para a Web 2

Aula 13

Evandro J.R. Silva

# Erros esperados vs. inesperados

- **Erro esperado** é condição prevista pelo domínio ou pelo protocolo.
  - O sistema sabe nomeá-la, e o cliente em geral pode corrigir a requisição, autenticar-se, esperar ou desistir. Entra no contrato.
- **Erro inesperado** é o que o *design* não tratou como caso de uso: defeito, invariante violada, dependência fora do ar, dado corrompido.
  - O cliente não tem o que corrigir. O destino é registro interno, correlação e uma resposta genérica.

# Erros esperados vs. inesperados

| Classe     | Quem pode agir       | O que o cliente recebe              | O que o servidor guarda    |
|------------|----------------------|-------------------------------------|----------------------------|
| Esperada   | O chamador, em geral | 4xx, código estável, detalhe seguro | Evento de negócio, se útil |
| Inesperada | O operador           | 5xx genérico e um identificador     | Exceção, causa, rastreio   |

- **Exemplos no mesmo endpoint:** considere `POST /contas`, que cria uma conta a partir de e-mail e senha
  - **Esperado:** e-mail ausente, senha curta, e-mail já cadastrado, formato inválido. O cliente enviou algo que o contrato recusa.
  - **Esperado, mas de infraestrutura negociada:** cota de tentativas excedida. O cliente pode esperar; o status `429` e o campo `Retry-After` fazem parte do contrato.
  - **Inesperado:** o banco recusou a conexão, um `null` escapou do serviço, o hash de senha lançou uma exceção não prevista. Nada disso é caso de uso.

# Erros esperados vs. inesperados

- A classificação se faz no ponto em que a falha nasce, e evita dois erros simétricos:
  - **Promover defeito a regra de negócio.** Ex.: um `NullPointerException` vira “dados inválidos” e o cliente recebe `4xx` por um bug.
  - **Esconder regra de negócio em `5xx`.** Ex.: e-mail duplicado vira “erro interno” e o cliente não sabe que deve escolher outro e-mail.
- De forma simplificada:
  - Se o cliente, lendo a documentação, poderia ter evitado a falha mudando a requisição, ou o momento do reenvio, a falha é **esperada**. Se só o time que opera o serviço pode corrigi-la, ela é **inesperada**.

# Códigos de status adequados

- O primeiro dígito indica a classe de resposta
  - 4xx afirma que o servidor entendeu a requisição e recusa cumpri-la por algo atribuível ao chamador, ao estado do recurso ou à política.
  - 5xx afirma que o servidor não cumpriu uma requisição aparentemente válida.
- Cliente e intermediário tratam as duas classes de modo diferente
  - 4xx em geral não se resolve com nota tentativa cega.
  - Vários 5xx e o 429 admitem *retry* com limite.

# Códigos de status adequados

Mapa mínimo para API de aplicação

| Situação                          | Status | Nota de uso                                                |
|-----------------------------------|--------|------------------------------------------------------------|
| Validação de sintaxe ou campo     | 400    | Corpo mal formado, tipo errado, obrigatório ausente        |
| Credencial ausente ou inválida    | 401    | Exite WWW-Authenticate. Não é “sem permissão”.             |
| Autenticado, mas sem direito      | 403    | A identidade é conhecida; ação proibida                    |
| Recurso inexistente neta URI      | 404    | Não usar para esconder 403 sem decisão explícita           |
| Conflito com o estado atual       | 409    | Duplicidade, versão desatualizada, transição ilegal        |
| Corpo bem formado, regra recusa   | 422    | Aceito na prática; 400 também vale se o contrato for único |
| Muitas requisições                | 429    | Retry-After quando a espera for conhecida                  |
| Defeito ou dependência caída      | 500    | Mensagem genérica. Detalhe só no log                       |
| Dependência não respondeu a tempo | 504    | Não mascarar como 400                                      |

# Códigos de status adequados

- Distinções importantes
  - 401 e 403.
    - 401: não sei quem você é, ou a prova de identidade falhou. 403: sei quem você é e esta ação não é permitida. Login com senha errada é 401. Usuário autenticado apagando recurso de outro, sem política de ocultação, é 403.
  - 404 e 403.
    - 404 diz que não há representação nesta URI. 403 diz que há uma recusa de autorização. Usar 404 para não revelar recurso alheio é uma decisão de segurança e tem de ser uniforme: a mesma URI inexistente e a URI alheia precisam ser indistinguíveis, inclusive no tempo de resposta quando isso importar.
  - 400, 409 e 422.
    - 400 para o que nem chega a ser uma requisição bem formada. 409 quando o recurso existe e o estado impede a operação — duplicidade, conflito de versão. 422 quando a sintaxe está correta e a semântica do domínio recusa o valor. Se a equipe padronizar tudo que é “entrada ruim” como 400, que o faça por escrito; o erro é misturar sem critério.
  - 500 e falha de cliente
    - Saldo insuficiente, cupom expirado e estoque esgotado não são 500. São recusas previstas. 500 reserva-se ao que o serviço não soube concluir.

# Mensagens úteis e seguras

- Útil para quem consome
  - Uma mensagem útil responde três perguntas, quando for seguro respondê-las: o que foi recusado, por que o contrato recusou, e o que o cliente pode fazer. Utilidade não é um parágrafo de desculpas. É estrutura:
    - **Código estável**, independente do idioma: `email_ja_cadastrado`, `campo_obrigatorio`.
    - **Mensagem humana**, curta, no idioma da API. Pode mudar sem quebrar o cliente, desde que o código permaneça.
    - **Erros por campo**, quando a falha é de validação. Um único “dados inválidos” obriga o usuário a caçar o problema.
    - **Identificador de correlação**, em toda resposta de falha. O suporte localiza o registro sem pedir print de pilha.

# Mensagens úteis e seguras

- Segurança para quem opera
  - O corpo de erro é uma interface pública. Tudo o que ele contém pode ser lido por um cliente hostil. O registro interno é outra interface, com outro público.
    - Não devolver rastreamento de pilha, consulta SQL, caminho de arquivo, nome de classe interna, *host* de banco ou variável de ambiente.
    - Não confirmar, na recuperação de senha ou no login, se o e-mail existe. A mensagem segura é a mesma nos dois casos: “se houver uma conta, enviaremos instruções”. O status também deve ser o mesmo.
    - Não incluir o valor rejeitado quando esse valor for segredo ou dado pessoal desnecessário. “Senha recusada” basta; a senha não volta no `eco`.
    - Não diferenciar, por texto ou por tempo óbvio, o 404 de recurso inexistente e o 404 usado para ocultar recurso de outro usuário, se a ocultação for a política escolhida.
    - Detalhe técnico vai para o log, associado ao mesmo identificador enviado ao cliente.

# Mensagens úteis e seguras

- Forma recomendada: problema HTTP
  - [RFC 9457](#) (*Problem Details for HTTP APIs*) oferece um tipo de mídia, `application/problem+json`, com campos previstos: `type`, `title`, `status`, `detail` e `instance`. Extensões da aplicação — código estável, erros de campo, correlação — entram no mesmo objeto. O `status` do corpo deve repetir o `status` da linha de resposta.

# Mensagens úteis e seguras

Exemplo de validação, esperado e útil

```
HTTP/1.1 400 Bad Request
```

```
Content-Type: application/problem+json
```

```
X-Request-Id: 7f3c1a
```

```
{
  "type": "https://api.exemplo.edu/erros/validacao",
  "title": "Requisicao invalida",
  "status": 400,
  "detail": "Dois campos nao atendem ao contrato.",
  "code": "validacao_falhou",
  "request_id": "7f3c1a",
  "errors": [
    { "field": "email", "code": "formato_invalido" },
    { "field": "senha", "code": "muito_curta", "detail": "Minimo de 10 caracteres." }
  ]
}
```

# Mensagens úteis e seguras

Exemplo de falha inesperada. O cliente recebe o mínimo; o operador recebe o resto no log da mesma requisição.

```
HTTP/1.1 500 Internal Server Error
Content-Type: application/problem+json
X-Request-Id: 7f3c1a
```

```
{
  "type": "https://api.exemplo.edu/erros/interno",
  "title": "Erro interno",
  "status": 500,
  "detail": "Nao foi possível concluir a operacao.",
  "code": "erro_interno",
  "request_id": "7f3c1a"
}
```

# Mensagens úteis e seguras

- O que não entra na mensagem, mesmo sendo verdadeiro
  - Exemplo 1
    - Ruim: `duplicate key value violates unique constraint "contas_email_key"`
    - Bom: `code = email_ja_cadastrado, status = 409`
  - Exemplo 2
    - Ruim: `NullPointerException at ContaService.java:88`
    - Bom: `code = erro_interno, status = 500`, request\_id no corpo e no log
  - Exemplo 3
    - Ruim: `Usuario nao encontrado` (no login)
    - Ruim: `Senha incorreta` (no login, ramo diferente)
    - Bom: `code = credencial_invalida` (mesmo corpo nos dois ramos)